

Un percorso di lettura attraverso docxwatermarker

Fabio F.G. Buono

2026

Versione 1

Distribuito sotto licenza CC BY 4.0

Questa guida propone un ordine per leggere il repository, per uno studente che incontra codice e specifica formale insieme per la prima volta, o per un docente in cerca di un esempio svolto. Rimanda agli altri documenti e lascia a loro il dettaglio. L'edizione inglese è *A reading path through docxwatermarker*.

1 Perché questo ordine

Molti corsi insegnano la logica di Hoare su un `while` di tre righe e insegnano il codice vero in progetti che di specifica formale non ne hanno da nessuna parte. Le due cose non si incontrano mai. Questo repository le mette in un posto solo, perciò la cosa che vale la pena seguire non è l'una o l'altra da sola ma il filo che le lega. Quel filo passa per un pezzo di codice, la proprietà che dovrebbe avere, e il test che fallisce quando le due si allontanano. Il percorso qui sotto lo segue tre volte, a tre livelli.

2 Primo giro: un'operazione, da cima a fondo

Si parte da una singola operazione della libreria e la si guarda da tutti e tre i lati prima di allargare.

1. **Il codice.** Apri `src/docxwatermarker/core.py` e leggi `Template.replace_image`. Prende un template e un'immagine e restituisce un nuovo template con una immagine sostituita. Nota che restituisce un oggetto nuovo e non muta mai l'originale, e che verso la fine chiama `ensure(...)` con un argomento `spec="I2"`.
2. **La specifica.** Apri `SEMANTICA.md` (o il composto `SEMANTICA.pdf`) e trova la tripla per `replace_image` e l'invariante I2, preservazione del nameset. Leggi la tripla di Hoare come un contratto. Date queste precondizioni, il template restituito soddisfa queste postcondizioni. L'invariante I2 è la proprietà che l'insieme delle entry dell'archivio non cambia per effetto dello scambio.
3. **Il legame.** Guarda di nuovo la chiamata `ensure(spec="I2")` nel codice. Quel `spec="I2"` è lo stesso I2 che hai appena letto nel documento. I due non sono connessi da un commento che si può corrompere. Condividono un identificatore. La sezione *Specifica assiomatica* del README spiega il meccanismo.
4. **Il guardiano.** Apri `tests/test_spec_crossref.py`. Legge ogni `spec=` dal codice e ogni invariante dal documento e controlla che i due insiemi siano in accordo. Rinomina I2 in un punto solo e questo test diventa rosso. Esegui la suite una volta così l'hai vista passare, e allora sai cosa significherebbe il suo fallimento.

3 Secondo giro: il design che rende semplici i contratti

Capita un'operazione, si legge perché tutta la libreria è fatta come è fatta, perché è la forma a rendere corta la specifica.

1. Le note *No XML manipulation* e *Template is immutable* nel README. La libreria tratta il `.docx` come uno ZIP e scambia i byte di una entry, e ogni operazione restituisce un nuovo template. Sono queste due scelte a far sì che la specifica possa parlare di funzioni pure su archivi, senza stato mutabile da considerare.
2. `src/docxwatermarker/_invariants.py`, le primitive `require` ed `ensure`. Sono i controlli design-by-contract che il codice porta a runtime, e l'argomento `spec=` è ciò che lega un controllo a una clausola. La lettura di accompagnamento è Meyer sul design by contract, citato nella specifica.
3. I preliminari di `SEMANTICA.md`, gli insiemi e le funzioni su cui le triple sono scritte. Leggili ora, dopo il codice, così i nomi astratti (*Archive*, *Template*) si attaccano agli oggetti concreti che hai già visto.

4 Terzo giro: il comando, come processo

La libreria è funzioni pure. Lo strumento da riga di comando è un processo, e ha bisogno di una semantica di natura diversa. È la parte che un corso mostra di rado, due stili di specifica formale su una sola base di codice, ciascuno dove si adatta.

1. `src/docxwatermarker/cli.py`, la funzione `cmd_stamp`. Leggila come una sequenza di fasi, risolvi la sorgente, apri il template, costruisci l'immagine, scambia, salva, e opzionalmente produci un PDF, ciascuna in grado di fermarsi con un codice d'uscita numerico.
2. `OPERAZIONALE.md` (o `OPERAZIONALE.pdf`), che modella quella funzione come un sistema di transizioni. Le configurazioni accoppiano una fase a uno stato, le configurazioni terminali sono i codici d'uscita, e le regole sono le fasi che hai appena letto nel codice. Confronta la regola di ogni fase con il tratto corrispondente di `cmd_stamp`.
3. Il guardiano sui codici d'uscita, di nuovo in `tests/test_spec_crossref.py`, nella parte operativa. Estrae i codici che il comando restituisce e li controlla contro i codici che il documento operativo tabula. È la controparte operativa del legame `spec=`, la stessa idea applicata a un tipo di specifica diverso.
4. La sezione *Sintassi dei comandi e riconoscimento del formato* nello stesso documento operativo. Descrive la riga di comando con una grammatica BNF e il controllo di formato della libreria con un automa a stati finiti, scritto come tabella di transizione. Leggila anche per un secondo motivo. Dice in chiaro perché le funzioni pure della libreria non sono disegnate come automi, e perché un riconoscitore interno lo è. La scelta di quale strumento si adatta a quale oggetto è parte della lezione.

5 Cosa portarne via

Alla fine i tre file che questa guida ha percorso, il codice, la specifica e i test, dovrebbero leggersi come un unico pezzo connesso. Il codice fa il lavoro, la specifica dice cosa il lavoro garantisce, e i test si rifiutano di lasciare che i due siano in disaccordo. La lezione non è una singola tripla o regola. È che una specifica tenuta onesta da un test è una descrizione viva, dove un commento o un riferimento a numero di riga si sarebbe corrotto la prima volta che il codice si è spostato. Un piccolo lavoro di ingegneria può portare tutto questo, e questo è stato costruito per mostrarlo.